

CampF1R3

A small, multi-threaded, WASM-portable in-memory tuple store
that serves as an RSpace

Specification, version 0.1 (draft for review)

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

17 September 2026

Abstract

CampF1R3 is a small, highly performant, multi-threaded, in-memory key-value database written in Rust, cross-compileable to WebAssembly, whose sole purpose is to serve as an RSpace: the tuple store against which a rho calculus reducer materialises sends and receives and from which it obtains communication events. This document specifies CampF1R3’s semantics, data structures, operations, concurrency protocol, determinism profiles, cost-accounting hooks, portability constraints, public interface, and conformance regime. The semantics is taken from the RSpace denotation of the rho calculus — tables $T_K(R) = K \multimap (M_{\text{fin}}(R) \uplus M_{\text{fin}}([N]R))$, parallel composition as key-wise multiset union, reaction as the zip of dual bags — extended to the joins, patterns, persistence, peeks and guards that the F1R3FLY reducer actually requires. The design lever is the factoring of RSpace into a store on keys and a reflection on values: CampF1R3 is the store, parametric in its key object, its inner collection, and its matcher, and it knows nothing about processes. Parsing and normalisation are out of scope and are treated in a separate specification.

Contents

1	Introduction	3
1.1	Goals	3
1.2	Non-goals	4
1.3	Conformance language	4
2	Semantic model	4
2.1	The table, and why it is the specification	4
2.2	From tables to what the reducer needs	4
2.3	State	5
2.4	Redexes and quiescence	5
2.5	Operations as cuts	6
2.6	Correspondence with the calculus	7
2.7	Footprints and independence	7
3	Architecture	8
3.1	Crate family	8
3.2	The three parameters	8

4	Core traits	9
4.1	Values supplied by the caller	9
4.2	Matcher	9
4.3	Collections	9
4.4	Key object	10
5	Data structures	10
5.1	Rows and shards	10
5.2	Join groups and their home	11
5.3	Records and order keys	11
5.4	Matching strategy	11
6	Operations	12
6.1	Mutating operations	12
6.2	Reads and administration	13
7	Concurrency	13
7.1	Model	13
7.2	Lock protocols	13
7.3	Lock implementation by target	14
7.4	Reads without writers' locks	14
7.5	Optional optimistic matching	15
8	Checkpoints and the event log	15
8.1	Soft checkpoints	15
8.2	Event log	15
9	Determinism and replay	15
10	Cost accounting	16
11	Storage variants	17
11.1	Collections	17
11.2	Key objects	17
11.3	Qualifiers	17
12	Portability and WebAssembly	18
12.1	Targets	18
12.2	Dependencies	18
12.3	The WebAssembly API	18
13	Rust interface	18
14	Integration with flr3node	19
15	Performance	20
15.1	Complexity	20
15.2	Benchmarks and targets	20
16	Conformance and testing	21

1 Introduction

The RSpace of `f1r3node-rust` (crate `rspace++`, branch `feature/cost-accounted-rho`) is roughly 21 000 lines of Rust, of which the in-memory tuple store — the part the reducer actually talks to during evaluation — is a few thousand. The remainder is the radix history trie, LMDB persistence, the merger, and replay. The whole crate depends transitively on `heed` (LMDB, C code), `tokio`, and `rayon`, and its hot paths call `std::time::Instant::now`, which panics on `wasm32-unknown-unknown`. None of that is required to be an RSpace.

CampF1R3 is the RSpace with everything that is not the store removed. It is specified here as a standalone crate family with three commitments:

1. it is exactly the store monad together with the zip, generalised to what the reducer needs, and nothing else (Section 2);
2. it runs unchanged on native targets and on WebAssembly, with or without threads (Section 12);
3. it plugs into the existing reducer through the existing type parameters $C, P, A, K = \text{Par}, \text{Bind-Pattern}, \text{ListParWithRandom}, \text{TaggedContinuation}$, so the `f1r3node` parser, normaliser and spatial matcher can be reused once they are extracted (Section 14).

Sources.

- Semantic basis: the RSpace denotation of *Quoting is Colour-Swap*, §5 (`publications/denotational-semantics-for-rho/knot-rho.tex`).
- Key-object polymorphism: *Paths are Subspaces* (`publications/polymorphic-rspace/paths-subspaces.tex`).
- Operational surface and storage variants: the approved FIP *Reifying RSpaces* (`FIPs/approved/2025-09-26-Reifying-RSpaces`).
- Interface to be served: `ISpace` and `Match` in `rspace++`, `f1r3node-rust` commit 619beb4.

1.1 Goals

G1 Small.

The core crate is at most about 3 000 lines of non-test Rust and has no mandatory third-party dependency outside a short allowlist (Section 12.2).

G2 Fast.

The common operations — a send or receive on a single channel that belongs to no join — touch one shard, take one lock, perform no allocation beyond the stored record, and never hash or serialise a payload.

G3 Multi-threaded.

Operations whose footprints are disjoint run in parallel; throughput on disjoint channels scales with the number of shards up to the number of hardware threads.

G4 Portable.

The core builds for `wasm32-unknown-unknown` (single threaded), `wasm32-unknown-unknown` with atomics, and `wasm32-wasip1-threads`, with identical observable semantics.

G5 Conformant.

Every operation preserves the quiescence invariant of Section 2, and the store is linearisable.

G6 Reusable.

The store is generic in channel, pattern, datum and continuation types, and in the matcher, so the `f1r3node` types plug in without conversion.

1.2 Non-goals

CampF1R3 does *not* parse, normalise, substitute, or evaluate processes; it does not run continuations; it does not persist to disk, maintain a history trie, compute state roots compatible with `rspace++`, merge concurrent blocks, gossip, or take part in consensus. It does not spawn threads: parallelism is supplied by the caller. Vector-database channels and quantale semantics for sequential processes (both discussed in the FIP) are out of scope for version 1.

1.3 Conformance language

The keywords `MUST`, `MUST NOT`, `SHOULD` and `MAY` are used in the sense of RFC 2119. Numbered *Requirements* are normative. *Remarks* are explanatory. *Obligations* are proof or validation tasks that the design relies on but that this document does not discharge. *Open decisions* are choices deferred to review; each states the default this draft adopts.

2 Semantic model

2.1 The table, and why it is the specification

In the `RSpace` denotation, a process modulo structural congruence is sent to a table

$$T_K(R) = K \multimap (M_{\text{fin}}(R) \uplus M_{\text{fin}}([N]R)),$$

a finite partial map from keys to polarity-homogeneous bags: at each key either a bag of output payloads or a bag of input continuations. Parallel composition is key-wise multiset union $\uplus_K$, so the denotation is a commutative-monoid homomorphism and the parallel laws of structural congruence hold as literal equalities of tables. Reaction is *eager*: a key carrying both polarities is a redex, and firing it zips the two bags, applying each matched continuation to the quoted payload. `RSpace` is the store monad $K \multimap M_{\text{fin}}(-)$ together with this zip, presented as a distributive law — a bialgebra.

Two facts from that account govern the whole of CampF1R3.

Remark 2.1 (The factoring). `RSpace` is a reflection structure on *values* glued at the table to a store monoid on *keys*. Changing the key object, or the collection at each key, leaves the reflection law untouched. CampF1R3 implements only the store-and-zip factor. Quoting and dereference are the caller’s business, carried inside the opaque types C , A and K .

Remark 2.2 (No implicit interaction). The store never reacts with itself. A reaction is triggered only by a cut: the arrival of one send or one receive, co-located with the store. In CampF1R3 every mutating operation is exactly such a cut, and nothing else ever fires.

2.2 From tables to what the reducer needs

The table above is single-channel, patternless and linear. The F1R3FLY reducer needs five generalisations, each of which CampF1R3 `MUST` support:

1. **joins** — a receive listens on an ordered group of channels $g = (c_1, \dots, c_n)$ and fires only when every position is satisfied;

2. **patterns** — a redex requires a successful match, not mere co-location;
 3. **persistence** — persistent sends and receives ($\leq$, replicated output), the store-level face of the $\mathbb{N} \cup \{\omega\}$ completion forced by replication;
 4. **peeks** — positions of a join that read a datum without consuming it;
 5. **guards** — a **where** clause evaluated over all bindings of a join after spatial matching succeeds.
- With patterns, polarity homogeneity is no longer the normal-form condition: a channel may legitimately hold both data and continuations that simply do not match. The normal form becomes *quiescence*, the “no match remains” invariant of the FIP.

2.3 State

Fix sets C (channels, with decidable equality), P (patterns), A (data), and K (continuations), and a *matcher* consisting of

$$m : P \times A \rightarrow A \quad \text{and} \quad \gamma : K \times A^* \rightarrow \{\top, \perp\},$$

where $m(p, a)$ is defined when a matches p , returning the binding result, and γ is the guard. Let $\pi \in \{\text{lin}, \text{pers}\}$ be a persistence flag.

Definition 2.3 (Records). A *datum* is $d = (a, \pi, h)$ with $a \in A$ and h its content hash. A *waiting continuation* is $w = (\vec{p}, k, \pi, E, h)$ with $\vec{p} = (p_1, \dots, p_n) \in P^n$, $k \in K$, and $E \subseteq \{1, \dots, n\}$ the peek positions.

Definition 2.4 (Store state). A state is $\Sigma = (D, W, I)$ where

$$D : C \rightarrow \text{Bag}(\text{datum}), \quad W, I : C^+ \rightarrow \text{Bag}(\text{waiting continuation}),$$

with W the ordinary and I the installed continuations, every continuation at group g having $|\vec{p}| = |g|$. The join index is derived, not stored independently: $J_\Sigma(c) = \{g \in \text{dom}W \cup \text{dom}I \mid c \in g\}$.

For the bag instance, states form a commutative monoid under key-wise $\uplus$ with the empty state as unit, which is the table monoid of the source paper extended by the group-keyed component.

2.4 Redexes and quiescence

Definition 2.5 (Candidate). Let $w = (\vec{p}, k, \pi, E, h)$ wait at $g = (c_1, \dots, c_n)$. A *candidate* for w is a tuple of data occurrences $\vec{d} = (d_1, \dots, d_n)$ with d_i an occurrence in $D(c_i)$ such that

1. occurrences are pairwise distinct (if $c_i = c_j$ with $i \neq j$, then d_i and d_j are different occurrences, though they may be equal values);
2. $b_i = m(p_i, a_i)$ is defined for every i ;
3. $\gamma(k, \vec{b}) = \top$.

$\text{Redex}(\Sigma)$ is the set of triples $(g, w, \vec{d})$ with $\vec{d}$ a candidate for w .

Definition 2.6 (Quiescence). Σ is *quiescent*, $Q(\Sigma)$, when $\text{Redex}(\Sigma) = \emptyset$.

Requirement 1 (Quiescence invariant). Every state observable between completed operations MUST be quiescent with respect to the configured matching strategy (Section 5.4).

2.5 Operations as cuts

We write $\Sigma +_c d$ for Σ with occurrence d added to $D(c)$, $\Sigma +_g w$ similarly, and $\Sigma - x$ for removal of an occurrence. A choice of redex is delegated to an *oracle* ω (Section 9): this is precisely the pairing nondeterminism of the source paper, localised at one cut.

Definition 2.7 (Consumption of a redex). For $(g, w, \vec{d})$ with $w = (\vec{p}, k, \pi, E, h)$, set

$$\text{fire}(\Sigma, g, w, \vec{d}) = \Sigma - \{w \mid \pi = \text{lin}\} - \{d_i \mid d_i.\pi = \text{lin} \text{ and } i \notin E\},$$

returning the *comm result* $\kappa = (k, \pi, g, \vec{p}, (b_i, a_i, d_i.\pi)_i)$: the continuation, its persistence, the group, the patterns, and for each position the binding result, the removed datum, and the datum's persistence.

The three mutating operations are given by the following rules, each read on a quiescent input state Σ .

$$\begin{array}{c} \frac{d = (a, \pi, h) \quad \nexists (g, w, \vec{d}) \in \text{Redex}(\Sigma +_c d) \text{ using } d}{\text{produce}(c, a, \pi) : \Sigma \Longrightarrow \Sigma +_c d, \text{ None}} \text{PRODUCE-STORE} \\[10pt] \frac{d = (a, \pi, h) \quad (g, w, \vec{d}) = \omega(\{r \in \text{Redex}(\Sigma +_c d) \mid r \text{ uses } d\})}{\text{produce}(c, a, \pi) : \Sigma \Longrightarrow \text{fire}(\Sigma +_c d, g, w, \vec{d}) \ominus d, \text{ Some}(\kappa)} \text{PRODUCE-COMM} \\[10pt] \frac{w = (\vec{p}, k, \pi, E, h) \quad \nexists \text{ candidate for } w \text{ in } \Sigma}{\text{consume}(g, \vec{p}, k, \pi, E) : \Sigma \Longrightarrow \Sigma +_g w, \text{ None}} \text{CONSUME-STORE} \\[10pt] \frac{w = (\vec{p}, k, \pi, E, h) \quad \vec{d} = \omega(\text{candidates for } w \text{ in } \Sigma)}{\text{consume}(g, \vec{p}, k, \pi, E) : \Sigma \Longrightarrow \text{fire}(\Sigma +_g w, g, w, \vec{d}) \ominus w, \text{ Some}(\kappa)} \text{CONSUME-COMM} \\[10pt] \frac{w = (\vec{p}, k, \text{pers}, \emptyset, h) \quad \nexists \text{ candidate for } w \text{ in } \Sigma}{\text{install}(g, \vec{p}, k) : \Sigma \Longrightarrow \Sigma +_g^I w, \text{ None}} \text{INSTALL} \end{array}$$

Here $\Sigma \ominus x$ removes occurrence x if it is still present (it may already have been removed by fire, or kept there as a peek). The arriving record is *never stored* when it reacts, even if persistent: its persistence is honoured by the caller re-issuing it, exactly as **ISpace** documents (“to make data stick, call produce until None”). **install** with an existing candidate is an error (installation happens at start-up), and installed continuations survive **clear** and checkpoint reversion.

Proposition 2.8 (Operations preserve quiescence). *If $\mathbf{Q}(\Sigma)$ and $\text{op} : \Sigma \Longrightarrow \Sigma'$ by any rule above, with ω drawing from the complete candidate set, then $\mathbf{Q}(\Sigma')$.*

Proof. For the STORE rules the premise says no redex of the extended state uses the new record; any other redex would already be a redex of Σ . For the COMM rules the new record is absent from Σ' , and Σ' is obtained from Σ by removing occurrences only. Removing data or continuations cannot create a candidate (candidacy is monotone in the bags: every candidate of a sub-state is a candidate of the state), so any redex of Σ' is a redex of Σ , contradicting $\mathbf{Q}(\Sigma)$. $\square$

Remark 2.9 (Why the arriving record is not stored on reaction). Proposition 2.8 depends on it. If a persistent arriving datum were stored after reacting with one continuation, a second waiting

continuation matching the same datum would be a live redex in the resulting state. The re-issue obligation is the price of keeping every operation a single cut, and is what lets the store return at most one comm result per call.

Requirement 2 (Re-issue contract). A caller that receives $\text{Some}(\kappa)$ from an operation whose arriving record was persistent MUST re-issue that operation after scheduling the continuation, until it returns None , if it wants the record to persist. CampF1R3 MUST NOT re-issue on the caller's behalf.

2.6 Correspondence with the calculus

Let $\llbracket \Sigma \rrbracket$ read a state back as a process: the parallel composition of $c!(a)$ for each linear datum, of its replicated form for each persistent datum, and of the corresponding (possibly joined, possibly replicated) receive for each continuation. For the bag instance with a matcher that is the reducer's own spatial matcher:

Proposition 2.10 (Store rules are homomorphic). $\llbracket \Sigma +_c d \rrbracket \equiv \llbracket \Sigma \rrbracket \mid c!(a)$ and $\llbracket \Sigma +_g w \rrbracket \equiv \llbracket \Sigma \rrbracket \mid \text{for}(\vec{p} \leftarrow g) k$. *Consequently two non-reacting operations commute: the resulting state is independent of their order.*

Obligation 1 (Comm rules are single reduction steps). Show that PRODUCE-COMM and CONSUME-COMM correspond to one COMM step of the joined, patterned calculus on $\llbracket \Sigma \rrbracket \mid$ arriving term, up to the re-issue contract and up to the resource bisimilarity that forgets the oracle's pairing. This extends the bialgebra obligation of the source paper from single-channel patternless tables to joins, patterns, persistence, peeks and guards.

Remark 2.11 (Non-bag collections). Proposition 2.10 holds *only* for bags. With queues, stacks or cells at a channel, the store monoid is no longer commutative and parallel composition can no longer denote key-wise union. Such collections are therefore offered only under the caveats of Section 11.

2.7 Footprints and independence

Concurrency rests on a single lemma, whose content is that the store's nondeterminism is local to channels.

Definition 2.12 (Footprint). The footprint of an operation on state Σ is the set of channels whose bags it may read or write:

$$\text{fp}_\Sigma(\text{produce}(c, \dots)) = \{c\} \cup \bigcup J_\Sigma(c), \quad \text{fp}_\Sigma(\text{consume}(g, \dots)) = \text{fp}_\Sigma(\text{install}(g, \dots)) = \text{set}(g).$$

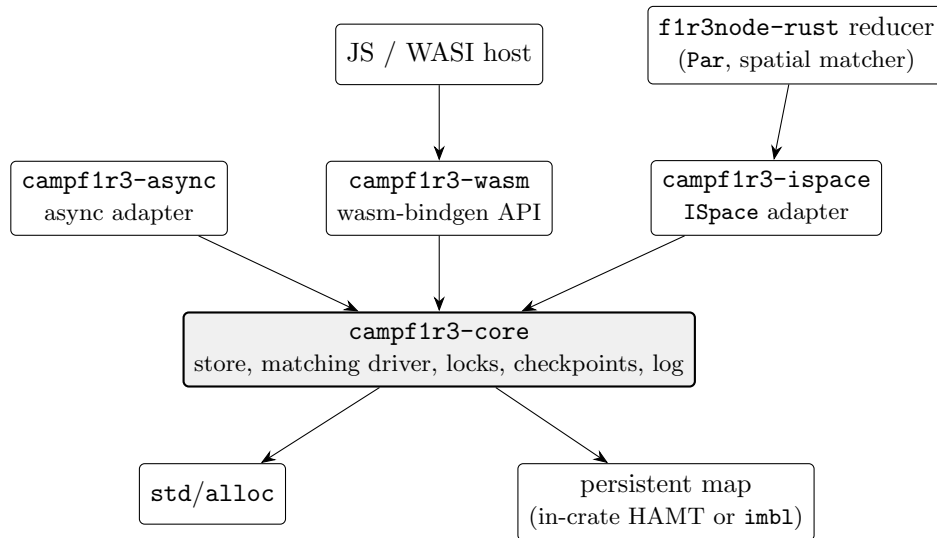
Lemma 2.13 (Independence). *If $\text{fp}_\Sigma(o_1) \cap \text{fp}_\Sigma(o_2) = \emptyset$, then executing $o_1; o_2$ and $o_2; o_1$ from Σ yields the same final state and the same results, for any oracle that depends only on the candidate set.*

Proof sketch. Neither operation can change the other's footprint. A consume on g adds only a group g , which cannot enter $J(c)$ for a channel $c \notin g$. A produce on c adds no group. Each operation's candidate set is computed from bags inside its footprint, and each writes only inside its footprint, so neither sees the other's effect. $\square$

Lemma 2.13 is the Mazurkiewicz independence relation of the store. It justifies sharding (Section 7): operations whose footprints map to disjoint sets of shards commute, so they may run in parallel, and any interleaving is equivalent to a serial one.

3 Architecture

3.1 Crate family



`campf1r3-core`

The store. It is synchronous, spawns no threads, uses no clocks and no randomness, and contains everything normative in Sections 2–10.

`campf1r3-async`

A thin `async` facade over the core, for callers whose reducer is asynchronous. It MUST NOT add semantics.

`campf1r3-wasm`

`wasm-bindgen` bindings exposing a byte-level instance of the core (Section 12.3).

`campf1r3-ispac`

An implementation of `rspace++`’s `ISpace` for the `f1r3node` types, without history. It lives in the `f1r3node-rust` workspace, not in the `CampF1R3` repository, so the core never depends on models.

3.2 The three parameters

Following Remark 2.1 and the FIP’s outer/inner split, the core is generic over three orthogonal choices:

Parameter	Role	Version 1 instances
Key object	How channels address rows; what “co-located” means	<code>Flat</code> (required), <code>Path</code> (optional)
Collection	The bag at each row, for data and for continuations	<code>Bag</code> (required); <code>Set</code> , <code>Cell</code> , <code>Queue</code> , <code>Stack</code> (optional)
Matcher	m and γ of Section 2	caller-supplied; <code>ExactBytes</code> reference matcher

Each parameter is a trait, and the store is monomorphised over it, so choosing a parameter costs no dynamic dispatch. A registry of named instances in the FIP style (`rho:space:HashMapBagSpace, ...`) belongs to the reducer’s system processes, not to the core.

4 Core traits

4.1 Values supplied by the caller

```
/// 32-byte content hash. Computed once by the caller (for f1r3node:
/// Blake2b-256 of the canonical serialisation, already present in the
/// Produce/Consume event records). The core never hashes payloads.
#[derive(Copy, Clone, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub struct Hash32(pub [u8; 32]);

pub trait Keyed {
    fn content_hash(&self) → Hash32;
}

pub trait Channel: Keyed + Eq + Clone + Send + Sync + 'static {}
pub trait Datum: Keyed + Clone + Send + Sync + 'static {}
pub trait Pattern: Clone + Send + Sync + 'static {}
pub trait Cont: Keyed + Clone + Send + Sync + 'static {}
```

Requirement 3 (Hash discipline). The core MUST NOT compute a hash of C , A , P or K values. All keying, sharding and ordering MUST derive from `content_hash`. Equal values MUST have equal hashes. Channel equality MUST be decided by `Eq` after hash equality, so a hash collision cannot merge two channels.

Remark 4.1. In `rspace++`, `shuffle_with_index` serialises every candidate with `bincode` and hashes it with Blake2b on *every* produce and consume, and shard and stripe selection use `std`'s `DefaultHasher`, whose output the standard library does not promise to keep stable. Requirement 3 removes the first cost entirely, and the second portability hazard.

4.2 Matcher

```
pub trait Matcher<P, A, K>: Send + Sync + 'static {
    ///  $m(p, a)$ : Some(binding result) when  $a$  matches  $p$ .
    fn get(&self, p: &P, a: &A) → Option<A>;
    ///  $\gamma(k, \text{bindings})$ : the where-clause guard, run after every
    /// position has matched. Default: no guard.
    fn check_commit(&self, _k: &K, _bound: &[A]) → bool { true }
}
```

This is `rspace++`'s `Match` trait verbatim, so the `f1r3node` spatial matcher implements it unchanged.

Requirement 4 (Matcher purity). A matcher MUST be a pure function of its arguments: deterministic, free of side effects, and MUST NOT call back into the store. It is invoked while shard locks are held (Section 7); a re-entrant call would deadlock.

4.3 Collections

```
pub trait Collection<T>: Clone + Default + Send + Sync {
    /// Insert an occurrence, returning its stable occurrence id.
    fn put(&mut self, item: T, key: OrderKey) → OccId;
```

```

fn remove(&mut self, id: OccId) → Option<T>;
/// Occurrences eligible to react, in the order the oracle sees them.
/// A Bag yields all; a Queue yields only its head; a Cell its one item.
fn eligible(&self) → impl Iterator<Item = (OccId, &T)>;
fn len(&self) → usize;
/// Algebraic laws the collection satisfies; see Section 9.
const COMMUTATIVE: bool;
const IDEMPOTENT: bool;
}

```

4.4 Key object

```

pub trait KeyObject<C: Channel>: Send + Sync + 'static {
/// Row key of a channel. Flat: its content hash.
fn row_key(c: &C) → Hash32;
/// Rows that a send on 'c' may meet, beyond its own row.
/// Flat: none. Path: rows on the route to and below 'c'.
fn comparable_rows(c: &C) → SmallRowSet;
}

```

5 Data structures

5.1 Rows and shards

The store is an array of S shards, S a power of two fixed at construction (default 64 on native targets, 1 on single-threaded WebAssembly). Row key r lives in shard $\text{sh}(r)$, computed from the leading eight bytes of the key:

$$\text{sh}(r) = (\text{u64}_{\text{le}}(r_{0..8}) \cdot \phi_{64}) \gg (64 - \log_2 S),$$

a Fibonacci hash with the fixed odd constant $\phi_{64} = 0\text{x}9\text{E}3779\text{B}97\text{F}4\text{A}7\text{C}15$. This is total, platform independent, and cheap.

```

/// DC: Collection<DatumRec<A>>, WC: Collection<ContRec<P, K>>
struct Shard<C, P, A, K, DC, WC> {
    lock: Lock, // Section 7
    rows: PMap<Hash32, Row<C, DC>>,
    groups: PMap<GroupKey, GroupRow<C, P, K, WC>>,
    version: u64, // bumped on every write
}

struct Row<C, DC> {
    channel: C,
    data: DC, // occurrences of DatumRec<A>
    joins: SmallVec<[GroupKey; 2]>, // J(c), ordinary and installed
}

struct GroupRow<C, P, K, WC> {
    channels: SmallVec<[C; 2]>, // ordered; repeats allowed
    waiting: WC, // occurrences of ContRec<P, K>
    installed: SmallVec<[ContRec<P, K>; 1]>,
}

```

```
}

```

PMap is a persistent hash-array-mapped trie with structural sharing, so cloning a shard's maps is a reference-count increment (Section 8). `rspace++`'s current hot store already adopted this design (`imbl::HashMap` per shard) to make soft checkpoints $O(S)$. CampF1R3 keeps it. A build feature `no-checkpoint` MAY replace PMap with an ordinary open-addressing map for callers that never checkpoint.

5.2 Join groups and their home

A group $g = (c_1, \dots, c_n)$ is keyed by $\text{GroupKey}(g) = H(\text{hash}(c_1) \parallel \dots \parallel \text{hash}(c_n))$, with H a fixed 64-bit mixing function over the concatenated channel hashes (computed by the core over hashes, not over channel values, which is consistent with Requirement 3), and equality confirmed by comparing the channel vectors. Order matters: patterns are positional.

The `GroupRow` lives in its *home shard*, the least shard index among $\{\text{sh}(\text{row}(c_i))\}$. Every member row records the group key in `joins`. Hence:

- $J(c)$ is read from c 's own row, under c 's own shard lock;
- every shard holding any part of g is in $\{\text{sh}(\text{row}(c_i))\}$, which is exactly the set a consume on g locks.

Empty rows and empty group rows MUST be removed eagerly, and a group key MUST be dropped from its member rows when the group row empties, so that the store's size is proportional to its live contents.

5.3 Records and order keys

```
struct DatumRec<A>    { value: A, persist: bool, key: OrderKey }
struct ContRec<P, K>  { patterns: SmallVec<[P; 2]>, cont: K,
                      persist: bool,  peaks: PeekSet, key: OrderKey }

#[derive(Copy, Clone, Eq, PartialEq, Ord, PartialOrd)]
struct OrderKey { hash: Hash32, seq: u64 }
```

`hash` is the record's content hash. `seq` is an arrival number taken from a per-space counter at commit time. `PeekSet` is a 64-bit mask; groups wider than 64 channels are rejected. A `Bag` keeps its occurrences sorted by `OrderKey` (a small sorted vector up to a threshold, a B-tree beyond it), so iteration in oracle order costs nothing extra per operation.

5.4 Matching strategy

Given a continuation w at g and the eligible occurrences at each position, the driver searches for a candidate (Definition 2.5).

Greedy (default).

For $i = 1, \dots, n$ take the first eligible occurrence at c_i , in order-key order, not already chosen, that matches p_i . If some position fails, or the guard rejects, the continuation has no candidate.

This is the strategy of `rspace++`'s `extract_data_candidates`.

Complete.

Backtracking search over the same order, returning the first candidate in lexicographic order-key order, or none if none exists.

Remark 5.1 (Greedy is incomplete). Greedy search misses candidates in two situations. One is a channel repeated in a group, with overlapping patterns, where the first choice at the earlier position steals the only datum that fits the later one. The other is a guard that rejects the greedy choice but would accept a later one. Under **Greedy**, Requirement 1 holds only relative to the strategy: no redex *discoverable by greedy search* remains. **Complete** restores Proposition 2.8 outright at a worst-case cost exponential in group width; group widths are small in practice.

Open decision 1 (Default matching strategy). Default: **Greedy**, for behavioural parity with **rspace++**. **Complete** is available per space. Review should decide whether guards (**where** clauses) oblige **Complete** for groups that carry a guard.

6 Operations

Every operation follows one skeleton: lock the footprint, consult the observer, search, then either fire or store, log, and unlock. Every write happens after the last point at which the operation can fail, so each operation is atomic.

6.1 Mutating operations

produce(*c*, *a*, *persist*).

1. Compute $r = \text{row_key}(c)$ and the datum occurrence d with order key ($\mathbf{a.content_hash}()$, $\perp$); its **seq** is assigned at commit.
2. Lock the footprint $\{c\} \cup \bigcup J(c)$ (protocol 7.2).
3. **observer.observe_produce**(...)?.
4. For each group $g \in J(c)$ in ascending **GroupKey** order, for each continuation at g — installed ones first, then waiting ones in order-key order — run the matching strategy with d presented as the *first* eligible occurrence at every position i with $c_i = c$. Accept a candidate only if it uses d . The first accepted candidate is offered to the oracle, which in the **Deterministic** profile takes it.
5. If a candidate is taken: **observer.observe_comm**(...)?; then remove the continuation if linear and not installed, remove each chosen stored datum that is linear and not at a peek position, log **Comm**, unlock, and return **Some(CommResult)**. d is not stored.
6. Otherwise assign **seq**, insert d into row r (creating the row if needed), log **Produce**, unlock, and return **None**.

consume(*g*, *patterns*, *k*, *persist*, *peeks*).

1. Reject if $|g| \neq |\vec{p}|$, if $|g| = 0$, if $|g| > 64$, or if a peek index is out of range.
2. Lock the shards of g in ascending order.
3. **observer.observe_consume**(...)?.
4. Run the matching strategy for the new continuation over the stored data.
5. If a candidate is found: **observer.observe_comm**(...)?; remove chosen linear non-peek data, log **Comm**, unlock, and return **Some**. The continuation is not stored.
6. Otherwise insert it into the group row in g 's home shard (creating the row and registering the group key in every member row if the group is new), log **Consume**, unlock, and return **None**.

install(*g*, *patterns*, *k*). As **consume** with **persist** = **true** and no **peeks**, except that a found candidate is an error (**InstallWouldReact**) and the continuation goes to **installed**.

6.2 Reads and administration

Operation	Contract
<code>get_data(c)</code>	Stored data at c , in order-key order. Takes c 's shard lock shared.
<code>get_waiting(g)</code>	Waiting and installed continuations at g .
<code>get_joins(c)</code>	$J(c)$ as channel vectors.
<code>remove_all_data(c), remove_all_continuations(g)</code>	Administrative removal; logs nothing.
<code>clear()</code>	Empties the store, then re-applies installed continuations.
<code>export()</code>	All rows and groups, sorted by key; deterministic.
<code>soft_checkpoint(), revert(cp)</code>	Section 8.
<code>take_log()</code>	Section 8.2.

Requirement 5 (Atomicity). If an operation returns `Err`, the store's state, counters and log **MUST** be exactly as before the call.

Remark 6.1 (Removing by index). `rspace++`'s `ISpace` exposes `remove_data_at(channel, index)`. `CampF1R3` does not expose positions: occurrence ids are stable, and the `ISpace` adapter translates positions to ids against the order-key order it presented.

7 Concurrency

7.1 Model

`CampF1R3` is a shared-memory store accessed by any number of caller threads. Every mutating operation holds all locks covering its footprint from before its first read to after its last write — strict two-phase locking at shard granularity. With Lemma 2.13 this gives the correctness criterion.

Requirement 6 (Linearisability). Every concurrent history of `CampF1R3` operations **MUST** be linearisable with respect to the sequential semantics of Section 2.5, with the linearisation point of each operation being its commit under locks.

Two operations that hold a common shard are serialised by it. Two operations with disjoint shard sets have disjoint footprints and therefore commute (Lemma 2.13), so any interleaving of them is equivalent to either serial order. What remains is that the footprint an operation locks is its true footprint at its linearisation point, and that locking cannot deadlock.

7.2 Lock protocols

Shards are totally ordered by index, and every protocol acquires locks in ascending index order.

Definition 7.1 (Consume and install). Lock $L_g = \{\text{sh}(\text{row}(c)) \mid c \in g\}$ in ascending order. L_g depends only on the arguments, so it is exact.

Definition 7.2 (Produce). Let $s_0 = \text{sh}(\text{row}(c))$.

1. Lock s_0 . Read $J(c)$ and compute $L = \{s_0\} \cup \{\text{sh}(\text{row}(c')) \mid c' \in g, g \in J(c)\}$.
2. *Fast path*: if $L = \{s_0\}$, proceed. This is the single-lock common case.

3. *Ascending path*: if $\min L = s_0$, lock the rest of L in ascending order while still holding s_0 , and proceed.
4. *Retry path*: otherwise release s_0 , lock L in ascending order, re-read $J(c)$ and recompute L' . If $L' \subseteq L$, proceed; otherwise release everything, set $L := L \cup L'$, and repeat. After three rounds, lock every shard.

Lemma 7.3 (Footprint stability). *While $s_0 = \text{sh}(\text{row}(c))$ is held, $J(c)$ and the channel vector of every group in $J(c)$ cannot change.*

Proof. $J(c)$ changes only when a group containing c is created (by `consume` or `install` on that group) or removed (when its last continuation is removed by a reaction or by administrative removal). Every such operation holds the shards of all the group's channels, s_0 among them. A group's channel vector is fixed at creation. $\square$

Proposition 7.4. *The protocols are deadlock free, and each operation locks a superset of its footprint's shards at its linearisation point.*

Proof sketch. All acquisitions are ascending: the ascending path acquires only indices above the one it holds, and the retry path holds nothing when it starts a round. Stability makes L computed under s_0 exact, and in the retry path the recomputation is made under s_0 as well. Livelock is bounded by the escalation to all shards. $\square$

7.3 Lock implementation by target

Build	Lock
native	<code>std::sync::Mutex<()></code> per shard, with shared reads through snapshots (below). A <code>parking_lot</code> feature MAY substitute.
wasm32, no atomics	A zero-cost cell. The type is <code>Sync</code> only under <code>cfg(not(target_feature = "atomics"))</code> , where no second thread can exist.
wasm32 + atomics, wasip1-threads	<code>std::sync::Mutex</code> , which uses <code>memory.atomic.wait32</code> .

Requirement 7 (Browser main thread). In builds with atomics, a contended lock blocks with `Atomics.wait`, which browsers forbid on the main thread. Blocking operations MUST be documented as worker-only, and the core MUST provide `try_`-prefixed variants that return `WouldBlock` instead of waiting.

Requirement 8 (No threads, no clocks, no randomness). The core MUST NOT spawn threads, read clocks (`std::time::Instant` panics on `wasm32-unknown-unknown`), or draw randomness. Metrics timing, if any, MUST go through an optional caller-supplied clock trait.

7.4 Reads without writers' locks

Read operations take the relevant shard lock only long enough to clone the shard's persistent maps, then read the clone with no lock held. For `get_data` this is a single reference-count increment.

7.5 Optional optimistic matching

Matching runs under locks, and a spatial matcher can be expensive, so a long match stalls its shards. The feature `optimistic` MAY be enabled:

1. snapshot the footprint shards (clone maps and record each shard’s `version`);
2. run the matching strategy on the snapshot, with no lock held;
3. lock the footprint, and commit the snapshot’s decision if every footprint shard’s version is unchanged; otherwise discard it and run pessimistically.

Validation is per shard, and therefore conservative. The result is still strict 2PL at commit, so Requirement 6 is unaffected.

8 Checkpoints and the event log

8.1 Soft checkpoints

`soft_checkpoint()` locks every shard in ascending order, clones each shard’s persistent maps ($O(S)$ reference-count increments), records the `seq` counter and the log watermark, and releases. `revert(cp)` locks every shard, replaces the maps, restores the counter, truncates the log to the watermark, re-applies the installed continuations, and releases. Both are global quiescent points.

Requirement 9. `soft_checkpoint` MUST cost $O(S)$ time and allocation, independent of the number of stored records.

There are no hard checkpoints. A caller that needs a state root MAY enable `digest`, which folds a caller-supplied hasher over `export()` in key order. Such roots are CampF1R3 roots, not `rspace++` history roots.

8.2 Event log

With the feature `log`, each committing mutation appends one event, stamped with a sequence number drawn from a per-space `AtomicU64` while its locks are held, to a buffer in its home shard (the least shard it holds):

$$\text{Produce}(h_c, h_a, \pi) \quad \text{Consume}(\vec{h}_c, h_k, \pi) \quad \text{Comm}(\text{Consume}, \overrightarrow{\text{Produce}}, E).$$

All hashes are caller-supplied content hashes.

`take_log()` reads the counter as a watermark M , then visits the shards in ascending order, draining each buffer’s events with sequence number below M , and finally sorts the result by sequence number. An operation whose number is below M was stamped while holding its home shard, before M was read, so the drain of that shard waits for it and sees the event. An operation that takes a shard after the drain passed it is stamped after M was read, so its number is at least M . The log therefore needs no global mutex, and its order is a valid linearisation order.

9 Determinism and replay

The source paper leaves the pairing of sends with receives as a relation. A store has to choose. CampF1R3 isolates the choice in the oracle ω and offers three profiles.

Free.

The oracle takes whatever candidate the strategy finds first, and collections MAY iterate in arrival order without sorting. Correct up to resource bisimilarity; fastest.

Deterministic (default).

Candidates are explored in the canonical order of Section 6: groups by `GroupKey`, installed before waiting, continuations and data by `OrderKey`. The first candidate wins.

Replay.

`rig(log)` loads an expected multiset of `Comm` events. The oracle accepts a candidate only if the continuation hash and datum hashes match an expected `Comm`, which is then decremented. An operation that would otherwise react with no matching expectation stores its record instead. `check_replay()` fails if expectations remain. This is `IReplayRSpace`'s contract. Under `Replay`, quiescence holds only relative to the expectations: a record stored while awaiting its recorded partner may coexist with an unrecorded match.

Requirement 10 (Deterministic profile). For a serial sequence of calls with equal arguments, in the `Deterministic` profile, results, `export()`, and the log **MUST** be bit-identical across targets, pointer widths, shard counts and thread counts. For a concurrent execution, they **MUST** equal those of the serial execution in log order.

Shard count does not affect determinism, because order keys and group keys are derived from content hashes rather than shard placement.

Remark 9.1 (Relation to `rspace++` ordering). `rspace++` orders candidates by the Blake2b hash of their `bincode` encoding, then by position. `CampF1R3`'s `OrderKey` is the same shape, with the hash supplied by the caller and the position replaced by a stable arrival number. Whether the two orders agree on every workload is Obligation 2.

Open decision 2 (Replay compatibility with `f1r3node`). Must `CampF1R3` replay logs produced by `rspace++`, and produce logs that `rspace++` replays? Default in this draft: the core guarantees Requirement 10 and the `Replay` profile over caller-supplied hashes. Bit-compatibility with `rspace++` event hashes and candidate order is a conformance target of the `campf1r3-ispace` adapter, validated by differential testing, not a core requirement.

10 Cost accounting

In the cost-accounted rho calculus, a communication is where resources are debited: located token stacks travel as data, and signature-keyed entries are simply further channels. The `COMM` is also the witness of `RSpace`'s distributive law. `CampF1R3` therefore hooks accounting exactly at the zip and nowhere else.

```
pub trait Observer<C, P, A, K>: Send + Sync + 'static {
    fn observe_produce(&self, c: &C, a: &A, persist: bool) → Result<(), Abort>;
    fn observe_consume(&self, g: &[C], ps: &[P], k: &K,
                      persist: bool, peeks: PeekSet) → Result<(), Abort>;
    fn observe_comm(&self, k: &K, k_persist: bool,
                   data: &[(&A, bool)]) → Result<(), Abort>;
}
```

This mirrors `rspace++`'s `RSpaceAccountingObserver`.

Requirement 11 (Accounting hooks). Observers are called under the operation's locks, before any write. An `Err` aborts the operation under the atomicity requirement. Observers **MUST NOT** call the store, but **MAY** mutate their own budget, which **SHOULD** be lock-free (for example an `AtomicU64`). Any policy about which data may match, such as `rspace++`'s refusal to match data carrying a cost stack, belongs to the matcher, not to the store.

11 Storage variants

11.1 Collections

Collection	Comm.	Idem.	Semantics and caveat
Bag	yes	no	Multiset. The only collection under which Proposition 2.10 holds. Required.
Set	yes	yes	Re-inserting an equal record (same content hash) is a no-op. Denotes $\mathcal{P}_{\text{fin}}$ rather than M_{fin} , so $x!(0) \mid x!(0) \equiv x!(0)$ at that channel.
Cell	–	–	At most one datum and one continuation; a second put is an error (CellOccupied).
Queue	no	no	FIFO; only the head is eligible. Parallel composition at the channel stops commuting.
Stack	no	no	LIFO; only the top is eligible. Same caveat.

Requirement 12 (Non-commutative collections). A space whose collection has **COMMUTATIVE** = **false** MUST be constructed with the **Seq** qualifier (below). Its determinism depends on caller-agreed arrival order, which CampF1R3 records but cannot establish across nodes.

11.2 Key objects

Flat keys each row by its channel’s content hash. It is required, and it is reading (0)/(1) of *Paths are Subspaces*: every theorem of the flat model transfers verbatim.

Path (optional) takes $C = \text{Name}^*$ and implements *cut-triggered subspace reaction*. The rows are a trie, and a send on p may meet a receive on any path comparable with p in the prefix order. The two dual rules are:

Out-deep a receive on p_1 meets a send on $p_2 = p_1 \cdot w$; the receiver binds the sealed, still-located sub-RSpace $@(w \triangleright Q)$.

In-deep a send on p_2 meets a receive on $p_1 = p_2 \cdot w$; the cut unpacks into a fresh cut $\text{for}(y \leftarrow w)P \mid Q$, which CampF1R3 returns to the caller as a comm result of kind **Descend**, not as a store mutation.

No-implicit-interaction (Remark 2.2) is preserved: the trie never reacts internally, and footprints become “the row and its comparable rows”, which **KeyObject::comparable_rows** supplies to the lock protocol.

Open decision 3 (Path semantics). The FIP’s PathMap example composes stores upward, so a send is also a send on every prefix, and it has no IN-DEEP rule. *Paths are Subspaces* insists on no implicit interaction and has both rules. Default: implement the latter, and reconcile the FIP before **Path** is stabilised.

11.3 Qualifiers

The FIP’s qualifiers are accepted at construction.

Default Concurrent. Included in **export** and **digest**.

Temp Concurrent. Excluded from **export** and **digest**, so the caller does not persist it.

Seq As **Temp**. The FIP’s static restrictions (the name is never sent and is used in at most one sequential process) are enforced by the reducer. CampF1R3 MAY additionally assert single-threaded access in debug builds.

12 Portability and WebAssembly

12.1 Targets

Target	Level	Notes
x86_64/aarch64 Linux, macOS	MUST	Reference platform.
wasm32-unknown-unknown	MUST	Single-threaded; $S = 1$; zero-cost locks.
wasm32-unknown-unknown with atomics	SHOULD	Needs <code>-C target-feature=+atomics,+bulk-memory</code> , nightly <code>build-std</code> , and a host serving COOP/COEP headers for <code>SharedArrayBuffer</code> ; workers supplied by the host.
wasm32-wasip1-threads	SHOULD	Stable target; for wasmtime and similar non-browser hosts.

Requirement 13 (Portable arithmetic). Identifiers, sequence numbers, counters and sizes that are observable or logged MUST be `u64` or narrower fixed-width types, never `usize`, so that 32-bit WebAssembly and 64-bit native builds agree bit for bit.

12.2 Dependencies

Requirement 14 (Dependency allowlist). `campf1r3-core` MUST build with only `std` (or `alloc`) and, at most, `smallvec` and a persistent-map crate (`imbl`, or an in-crate HAMT). Optional features MAY add `parking_lot`. It MUST NOT depend, directly or transitively, on `tokio`, `rayon`, `heed` or any C library, `getrandom`, `serde`, `bincode`, or any crate of the `f1r3node-rust` workspace.

Serialisation is excluded on purpose: the caller already owns a canonical encoding and supplies its hashes. An optional `serde` feature MAY derive encodings for `export` snapshots.

12.3 The WebAssembly API

`campf1r3-wasm` instantiates the core at $C = A = K = \text{Bytes}$, with patterns drawn from a small reference language (exact bytes, prefix, wildcard) and the `ExactBytes` matcher. Content hashes are computed at the boundary by a host-provided function or by the bundled `blake3` feature. This makes `CampF1R3` usable, and testable, in a browser before the parser and normaliser are ported. A `rholang` instance replaces the reference matcher once those components are available.

```
#[wasm_bindgen] impl Space {
    pub fn new(config: JsValue) → Result<Space, JsError>;
    pub fn produce(&self, ch: &[u8], data: &[u8], persist: bool)
        → Result<Option<CommResult>, JsError>;
    pub fn consume(&self, chans: Array, pats: Array, cont: &[u8],
        persist: bool, peeks: u64) → Result<Option<CommResult>, JsError>;
    pub fn try_produce(/* same */ → Result<TryResult, JsError>; // WouldBlock
    pub fn checkpoint(&self) → Checkpoint;
    pub fn revert(&self, cp: Checkpoint) → Result<(), JsError>;
}
```

13 Rust interface

```

pub struct Space<C, P, A, K, M, O, DC, WC, KO> { /* shards, config */ }

pub struct Config {
    pub shards: u32,           // power of two
    pub profile: Profile,      // Free | Deterministic | Replay
    pub strategy: Strategy,    // Greedy | Complete
    pub qualifier: Qualifier,  // Default | Temp | Seq
}

pub struct CommResult<C, P, A, K> {
    pub cont: K, pub cont_persist: bool,
    pub channels: SmallVec<[C; 2]>, pub patterns: SmallVec<[P; 2]>,
    pub bound: SmallVec<[Bound<A>; 2]>, // (binding, removed datum, persist)
    pub peeks: PeekSet,
    pub kind: CommKind,             // Apply | Descend (Path only)
}

impl<...> Space<...> {
    pub fn new(cfg: Config, matcher: M, observer: O) → Self;
    pub fn produce(&self, c: C, a: A, persist: bool)
        → Result<Option<CommResult<C, P, A, K>>, SpaceError>;
    pub fn consume(&self, g: &[C], ps: &[P], k: K, persist: bool,
        peeks: PeekSet) → Result<Option<CommResult<C, P, A, K>>, SpaceError>;
    pub fn install(&self, g: &[C], ps: &[P], k: K) → Result<(), SpaceError>;
    // try_produce / try_consume, reads, admin, checkpoints, log, replay ...
}

```

All methods take `&self`, and `Space` is `Send + Sync` whenever its type parameters are. The error type is a closed enumeration: `Aborted` (observer), `ArityMismatch`, `GroupTooWide`, `InstallWouldReact`, `CellOccupied`, `WouldBlock`, `ReplayMismatch`, and `QualifierViolation`.

14 Integration with flr3node

`campflr3-ispace` instantiates the core at `Par`, `BindPattern`, `ListParWithRandom`, `TaggedContinuation`, with the reducer's spatial `Matcher` and a bridge from `RSpaceAccountingObserver` to `Observer`. Content hashes are the Blake2b-256 hashes that `rspace++` already computes for its `Produce` and `Consume` events.

ISpace method	CampF1R3
<code>produce, consume, install</code>	Direct.
<code>get_data, get_waiting_continuations, get_joins</code>	Direct.
<code>remove_data_at, remove_all_*</code>	Position translated to occurrence id.
<code>create_soft_checkpoint, revert_to_soft_checkpoint</code>	Direct.
<code>take_event_log</code>	<code>take_log</code> , events re-encoded to <code>rspace++</code> records.
<code>rig, rig_and_reset, check_replay_data, is_replay</code>	Replay profile.
<code>set_accounting_observer</code>	Observer bridge (swappable through an <code>ArcSwap</code>).
<code>to_map</code>	<code>export</code> .
<code>create_checkpoint, reset, get_root</code>	Unsupported unless <code>digest</code> ; roots are not <code>rspace++</code> roots.

The adapter is what makes differential testing against `rspace++` possible, and lets the reducer run over `CampF1R3` for evaluation paths that do not need history, such as exploratory deploys, local evaluation, and tests.

15 Performance

15.1 Complexity

Let $n_c = |D(c)|$, $j = |J(c)|$, w_g the number of continuations at g , $\ell = |g|$, μ the cost of one matcher call, and S the shard count.

Operation	Locks	Time (Greedy, excluding observer)
<code>produce, no joins on c</code>	1	$O(\log n_c)$ insert
<code>produce, with joins</code>	$\leq \ell \cdot j$ shards	$O(\sum_{g \in J(c)} w_g \ell \mu \cdot \max_i n_{c_i})$ worst case
<code>consume on g</code>	$\leq \ell$	$O(\ell \mu \cdot \max_i n_{c_i})$
<code>get_data</code>	1, briefly	$O(1)$ snapshot
<code>soft_checkpoint</code>	S , briefly	$O(S)$
<code>take_log</code>	S , in turn	$O(e \log e)$ for e events

The worst cases are dominated by μ , the spatial matcher. The store’s own overhead per matcher call is $O(1)$ and MUST NOT include hashing, serialisation or allocation of payloads.

15.2 Benchmarks and targets

The benchmark suite MUST include:

1. single-channel produce/consume ping-pong (fast path);
2. N disjoint channel pairs across T threads, for scaling;
3. a hot contended channel with T threads;
4. joins of width 2 and 4 with overlapping members;
5. persistent receive with a produce storm (the re-issue loop);
6. checkpoint and revert every k operations;
7. the `rspace++` benches `bench_par_branches_channel_reuse` and `bench_par_reducer`, run through the adapter.

Requirement 15 (Performance targets). Absolute targets are set after baseline measurement of `rspace++`'s hot store on the reference platform, with the `rspace++` history layer disabled. Version 1 MUST not regress against that baseline on any benchmark, SHOULD scale benchmark 2 to within 80% of linear up to $\min(S, \text{cores})$, and MUST run benchmarks 1 and 6 on `wasm32-unknown-unknown` within a small constant factor of native. That factor is to be fixed at review.

16 Conformance and testing

1. **Reference model.** A separate, deliberately naive, single-threaded implementation of Section 2.5 (vectors of records, exhaustive candidate enumeration) of a few hundred lines. It is the executable specification.
2. **Model-based property tests.** Random operation sequences run against the core and the reference model, comparing results, `export()`, and quiescence after every step (**Complete** strategy), or discoverable quiescence (**Greedy**).
3. **Linearisability checking.** Concurrent histories recorded under stress are checked against the reference model with a Wing–Gong style checker.
4. **Lock-protocol model checking.** The produce protocol and log drain are checked with `loom`, which `f1r3node-rust` already uses under `formal/loom`.
5. **Cross-target determinism.** The same seeded operation traces run natively and under `wasm-bindgen-test`, with `export()` and logs compared bit for bit (Requirement 10).
6. **Differential testing.** The `rspace++` test suite runs through `campf1r3-ospace`, and randomised reducer workloads are compared against `rspace++` up to resource bisimilarity, and exactly where Decision 2 demands it.

Obligation 2 (Differential agreement). Establish, by the differential suite, whether **Deterministic** candidate order agrees with `rspace++`'s, and characterise any divergence.

Obligation 3 (Greedy parity). Confirm that **Greedy** as specified coincides with `rspace++`'s `extract_data_candidates` on every workload, including repeated channels within a group, peeks, and guards.

17 Summary of obligations and open decisions

Item	Status in this draft
Obligation 1	Comm rules as single COMM steps — the bialgebra obligation extended to joins, patterns, persistence, peeks, guards. Open.
Obligation 2	Candidate-order agreement with <code>rspace++</code> . By test.
Greedy parity	By test.
Default strategy	Greedy ; guarded groups may require Complete .
Replay compatibility	Core: deterministic and Replay over caller hashes; bit-compatibility is an adapter target.
Path semantics	<i>Paths are Subspaces</i> over the FIP example, pending reconciliation.
Performance constants	Fixed after baseline measurement.

Roadmap

1. Reference model and property-test harness.

2. Core with `Flat` and `Bag`, `Greedy` and `Complete`, pessimistic locking, `Deterministic` profile, soft checkpoints.
3. `wasm32-unknown-unknown` build and byte-level API; cross-target determinism tests.
4. Event log, `Replay` profile, observer; `campfir3-ospace` and differential testing against `rspace++`.
5. Threads on WebAssembly; the `optimistic` feature; benchmarks and performance targets.
6. Optional collections, `Path` key object, and qualifiers.